Next-Gen AI Compiler

Agents On The Control Plane

Agents Own Search · Orchestration · Synthesis.
Compilers Own Lowering · Legality · Measure · Fallback.

Ask: Is The Hybrid Prediction (~2027–31) The Right Bet For Your Roadmap?

Expert Briefing · ~30–35 min + Q&A · Prediction-first

Agenda

1. Trends · Keep Substrate
2. Verdict + Architecture
3. Roadmap Checkpoints
4. Commercial Blockers + Gaps
5. Technical Prediction (Techniques)
6. Stance · Ask · Appendix

Trend backdrop — two stacks converging

2018–22	2020–23	2023–24	2025–26	2027+
DL compilers	RL gyms	LLM enters IR	Agentic hybrid	?

Compilers for AI
mature · fragmented substrate

AI for compilers
rapid hybrid growth

Next-gen ≠ throw away LLVM/MLIR — make them **agent-addressable**.

Six active trends

A — Hybrid guidance
Free IR rewrite fragile
(mlirAgent $<$ identity)
Constrain actions: hints /
advisory metadata / pass lists
AgentCompile/CityU · HintPilot/ZJU · Meta LLM Comp. · mlirAgent/UCB

B — RL → agents
CompilerGym → tool agents
Heuristics as shippable C++
Workflow compile · freeze · place
Compiler-R1/ISCAS · Magellan/DeepMind · Flow-Compile/UMass+MIT

C — MLIR + Triton
PyTorch→Inductor→Triton
Also StableHLO→XLA/IREE
Peak often vendor libs / Tile
Meta · OpenXLA/Google · OpenAI Triton · NVIDIA

D — Kernel agents
KernelBench: correct *and* faster
One-shot often $<$20%; fusion hard
GEAK · KernelLLM · AgentCompile
AMD · Meta · CityU · KernelBench: Stanford/Princeton

E — Verify in the loop
Unit/golden → numerical
→ Alive2-class local formal
Weak on GPU races / FP noise
Alive2/UIUC lineage · LLM-VeriOpt · stacked admit

F — Broader object
Mid-decode diagnosis
FMware: prompts / agents / knobs
Agents as compiler engineers
Compiler.next/Queen's · Magellan/Google · Claude C/Anthropic

Keep substrate, change control plane

Preserve

deterministic lowering
library kernels
build-CI regressability
local formal checks

Adopt

advisory search
multi-agent orchestration
heuristic synthesis
profile / verifier feedback

Anti-pattern: replace opt/Inductor with unconstrained LLM codegen

Executive verdict

Agents reshape the control plane
more than they replace the data plane.

Control plane: search / orchestrate / synthesize

Data plane: lower / legality / measure / fallback

Compilers for AI
× AI for compilers

Hybrid LLM–compiler loops

4th job Tier A:
bring-up / codesign

Will not happen soon

- Unconstrained LLM replaces Inductor/opt
- Autonomous chip tape-out via compiler agents (C10)

Four agent jobs — architecture spine

(a) Online
propose → measure admit
CompileIQ / GEAK
ACCLAIM / HintPilot

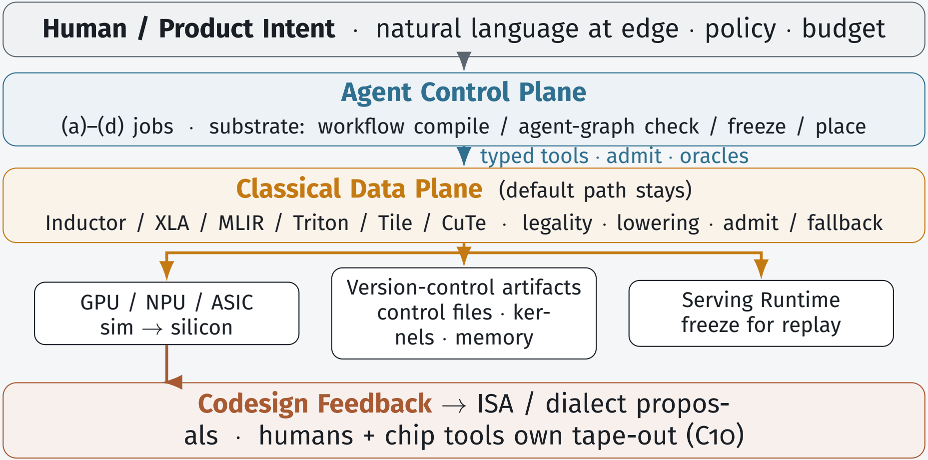
(b) Offline
evolve heuristics → C++ / MLGO
Magellan
AlphaEvolve

(c) Eng.
oracle-gated pull request / change
Archer

(d) Bring-up
coverage → performance sim + silicon
TritorX
KernelEvolve

Control plane jobs sit *above* classical lowering — not instead of it.

Architecture — Target Stack



Invariant: LLM guides search — does not silently define unchecked executable behavior.

Architecture evolution — component changes

	TODAY 2025–26	HORIZON A 2027–28	HORIZON B ∼2029–31
CTRL	**Ad-hoc agents** chat/tools · GEAK/ACCLAIM no typed admit	**Jobs (a–d) productized** online/offline/review/bring-up build-CI gated	**Control plane compiled** workflow · agent-graph · freeze audited agent graph
DATA	**Classical stack** MLIR/Triton/Inductor · GPU tools thin	**Agent-addressable** multi-DSL · typed tools · admit oracles on hot path	**Multi-backend fleets** control files / heuristics in version control classical lowering stays
CODE	**Sparse / manual** human bring-up dominant weak sim↔compiler loop	**Early feedback** sim + silicon → ISA/dialect proposals TritorX / KernelEvolve-class	**Steady pre-silicon** pre-silicon → bring-up → next chip not autonomous design (C10)
ART	**binaries** + sparse hints	**+ control files / kernels / C++**	**+ frozen agent workflows**

Components thicken left→right; data plane stays; agent graph becomes compiled & amortized.

Data plane — ∼6–7 abstraction bands (not one)

Must-have today (L1–L6)

L1	**Framework**	capture · dynamism
L2	**Portable graph**	StableHLO-class · shard ann
L3	**Mid-IR**	MLIR · layout · passes
L4	**Kernel DSL**	Triton/Helion/Tile/CuTe
L5	**Backend-ISA**	PTX · bring-up
L6	**Runtime-serve**	CUDA Graphs · KV

L0* CPU/LLVM optional · agents sit *above* bands via tools

L7* Fleet / cluster
maturing: place + collectives
today often L2–L3 + runtime

Not a new IR band
power/energy → objective + oracle
$/token · SLOs → control policy
safety → provenance / admit

Lean (A6/S6)
keep bands; agents unify *contracts*,
not one mega-IR

One universal cost model? — No (Horizon A)

Why not one model

Different legality / oracles per band

Choosing the level *is* the product

Cost models stay *local* (MLGO, Ansor)

Future passes need new measured labels

If bands do not consolidate

Agent compile schema (admit hash)

Typed tools / MCP-class servers

Dialect + oracle + objective plugins

Placement / fleet plugins (L7)

Size: local advisors often KB–MB · cross-band *proposer* ≈ 7B–70B+ IR LLM (still only a prior)
Bottleneck = labeled (program, action, HW, energy, SLO) tuples — not parameter count.
Bet: small per-band costs + optional large orchestrator — not one mega-cost-model for L1–L7.

E2E-optimal-seeking — reshape around F

Product fitness F (latency · energy · \$/token · quality · cluster) — Pareto/constrained, *not* $\sum$ local costs

E2E search controller — joint/bilevel policy · credit across bands · budget/stop/freeze

L2–L3	L4–L5	L6	L7*

bands = legality / lower surfaces · local cost = proposal prior only

Physical F-admit: serving A/B $\rightarrow$ pinned traces $\rightarrow$ energy/fleet · freeze or classical fallback

Claimed
e2e-optimal-*seeking* control plane
joint search + F-admit + freeze

Not claimed
unique closed-form global optimum
complexity bound $\neq$ stop trying

Merge $\neq$ replace — three levels

M1 Soft merge (Horizon A$\rightarrow$B)
e2e controller + admit/trace over band *tools* · data plane stays · agents do *not* replace compiler

M2 Search-surface merge (contested / partial ~2029–31)
fewer agent training IRs (C4) · lowers still multi-band · still hybrid

M3 Hard replace — *not* claimed thru Horizon B (~2031)
default path with *no* classical admit/fallback · only if C6-A settles

Perception trap: "I only talk to an agent" is often M1+freeze (classical lower still ran in CI).
Lean (A8 / C6-B): merge the *optimizer* under *F*; do not delete the execution substrate.

What ships / does not by 2028

Predicted to ship

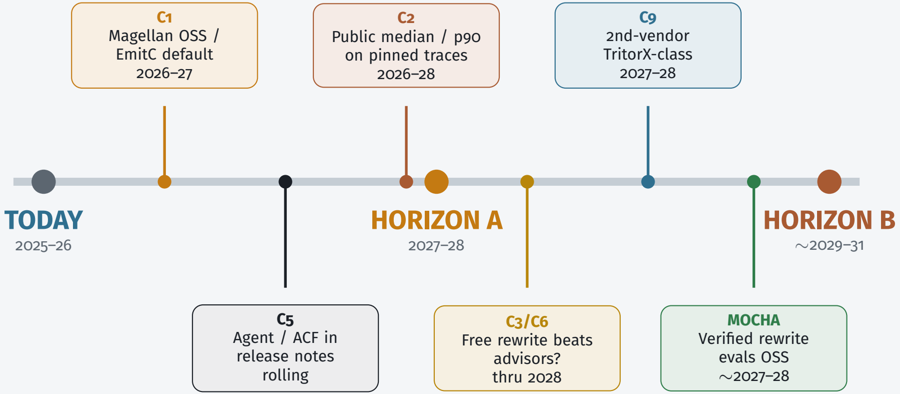
- Agent-addressable tool interfaces
- Hot-path specialize (not silent default-all)
- Magellan *and* MLGO both live
- Oracle pull-request review in serious orgs
- Coverage-first ASIC bring-up
- Triton-family primary; multi-DSL rising

Does *not* ship

- Unconstrained LLM replaces opt/Inductor
- One agent IR for all vendors
- Kernel agents uniformly beat eager on fusion ladders
- Autonomous microarch tape-out

Horizon A success = build-CI gated specialize + oracles + freeze artifacts

Roadmap Checkpoints — when the prediction changes



Falsifiable milestones — watch settlement signals; do not average disagreements.

Checkpoint C1 — heuristics vs neural advisors

A — Magellan path

evolve shippable C++ heuristics

Evolve shippable C++ (EVOLVE-BLOCK)

Offline coding agent *is* the control-plane output

Product = evolutionary coding agent + review

B — MLGO / EmitC path

MLGO = learned in-tree LLVM advisors

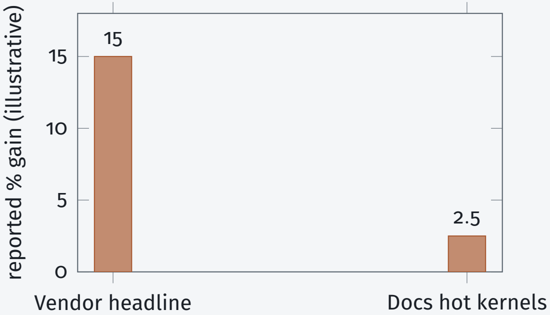
NN advisors stay in-tree

June 2026 PoR: inliner → Android/Fuchsia → Chrome

Agents train/feature NNs

Settlement: public Magellan llvm patches that displace MLGO *or* EmitC-MLGO becomes customer default
Watch LLVM syncs + OpenEvolve recipes quarterly through 2027

Checkpoints C2 + C5 — gains & default path



Not a meta-analysis — the tension to watch.

C2 "Agents become default" needs **median (p50) build-CI wins**, not best-kernel blogs.

p50 / p90 = 50th / 90th percentile of the gain distribution across builds.

C5 Online specialize (control files / hints) vs offline eng (Magellan/MLGO) — both may live; *which is the default flag?*

Settlement: pinned public traces + release notes listing agent / control-file workflows.

Checkpoints C3 / C6 / C9 / C10 — interface width & scope

C3 — free rewrite vs advisory
Flip: shared suite where free rewrite consistently wins
⇒ wide rewrite API vs narrow ACFs / hints (current lean)

C6 — replace vs control plane
Flip: production *default* path with no classical admit / fallback
⇒ agents replaced the compiler (survey rejects; hold hybrid)

C9 — coverage vs peak
Flip: TritorX → KernelEvolve ladder becomes a public playbook
⇒ SKU: coverage SLA first, then performance SLA

C10 — codesign vs auto tape-out
Flip: microarch primarily agent-proposed *and* compiler-oracle validated
⇒ enters chip-design tooling (today: agents stress kernels/IR only)

Commercial blockers — top 5

1. Oracles strong enough for money
   Fast-but-wrong · liability
2. Cost / replay / when agents run
   Nondeterministic $N compiles
3. Agent↔compiler contract
   Natural-language-only = demo
4. Distributional production evidence
   No default-path A/B
5. Ownership · supply chain · human review
   Unowned agent code in trusted base

Each blocker gets one slide — productization gaps, not research wishlist.

Blocker 1 — Oracles for money

Unit / golden / OpInfo — cheap, miss subtle bugs

Numerical tolerances — tolerance games

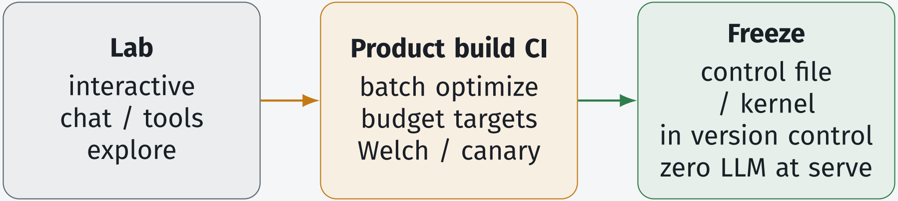
Alive2 / local honesty — strong local, weak GPU concurrency

Serving A/B — catches product · slow / attribution hard

Done: formal $\rightarrow$ shape-grid diff $\rightarrow$ statistical serving $\rightarrow$ staged rollout

Room question: who owns false negatives when admit passes and production miscompiles?

Blocker 2 — Cost, replay, when may the agent run?



Cache key: (IR hash, hardware, compiler ver, agent policy) · budget $/%gain

Falsifies commercially: “chat with compiler on every build” without spend/latency targets.

Blocker 3 — Agent↔compiler contract

A. Natural language + pasted logs — demo only

B. Structured admit traces — build CI / bill of materials

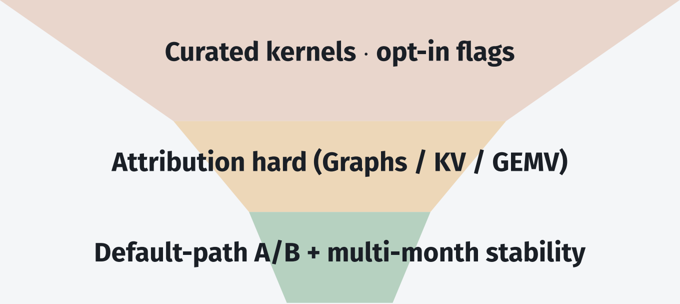
C. Typed tool interfaces — required action space

D. Hybrid — natural-language view over traces

```
admit_record = {graph_hash, hw_id, compiler_ver, action[],
oracle[], artifact_digest, policy_id}
```

Lean: **C + B**; Natural language is a view, not source of truth. ACF = portable compiler knobs.

Blocker 4 — Distributional production evidence



Marketing bar: median + 90th-percentile gains + cost-to-compile · persistent serving throughput, not single-kernel headlines

Blocker 5 — Ownership, security, human review

Trusted-base risk

agent kernels / heuristics enter trusted base

Ownership

CODEOWNERS signed provenance sandbox

Human-review capacity

agents $\uparrow$ drafts reviewers = bottleneck

Lean: human CODEOWNER + signed admit + sandbox · oracle auto-merge only for **narrow** action classes
Generic forge AI $\neq$ compiler-oracle review (C7)

Gap map — what blocks the prediction, not a wishlist

HIGH

- 4.1 prod evidence
- 4.2 correctness
- 4.3 cost/replay
- 4.4 cross-stack
- 4.10 benchmarks

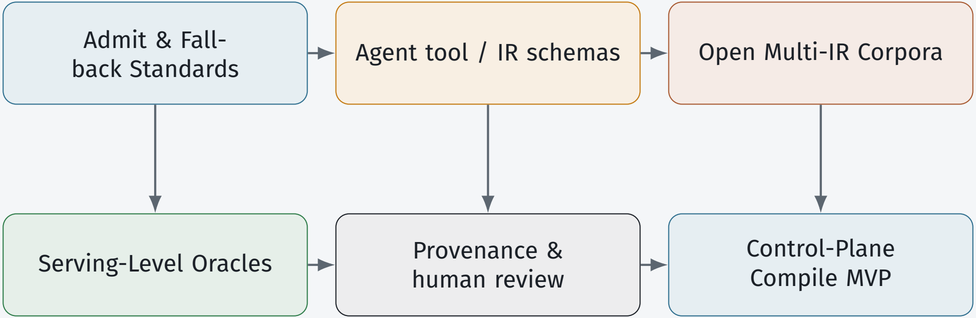
MED–HIGH

- 4.5 hardware-native interfaces
- 4.7 training data

MEDIUM

- 4.6 workflow compile
- 4.8 human-review process
- 4.9 security

Cross-Cutting Research Agenda



Near-term research themes → next: technique map T1–T10 (exists / missing / unlocks).

Technical Prediction — Accelerate The Roadmap

To settle checkpoints and ship Horizon A, enhance techniques *inside and outside* the compiler (T1–T10).

Within (T1–T5)
T1 Typed agent↔compiler interfaces
T2 Admit / fallback machinery
T3 Control files, hints, fingerprints + replay
T4 Heuristic hooks & in-tree advisors
T5 Dialect / ISA feedback sinks

Outside (T6–T10)
T6 Serving-level oracles & production A/B
T7 Open multi-IR corpora (+ negatives)
T8 Unified benchmark ladder
T9 Provenance, ownership, HITL process
T10 Agent-workflow compile / freeze / place

Enhancing only opt/Inductor/Triton internals is not enough. Next: exists · missing today · checkpoint unlocks.

Technical Prediction — Within The Compiler (T1–T5)

Exists · **Missing** · **Unlocks**

T1 Typed interfaces → **C3, C5, C6**
Exists: CompileIQ · ACCLAIM · mlir-opt-repl · FlashInfer Trace
Missing: portable schemas across MLIR · Triton · Tile · StableHLO

T2 Admit / fallback → **C6 hybrid**
Exists: AgentCompile · Archer · TritonRL verifiers · FlashInfer-Bench
Missing: shared admit *product* + trusted deterministic fallback

T3 Control files + replay → **C2, C5**
Exists: CompileIQ ACFs · FlashInfer Trace + `apply()`
Missing: content-addressed keys; golden replay on model upgrade

T4 Heuristic hooks / advisors → **C1**
Exists: Magellan / AlphaEvolve · MLGO · EmitC PoR
Missing: settled Magellan vs MLGO *default* on named apps

T5 Dialect / ISA sinks → **C9, C10**
Exists: TritorX · KernelEvolve · Ascend diagnosis
Missing: first-class change-*proposal* surfaces (not auto tape-out)

Technical Prediction — Outside The Compiler (T6–T10)

Exists · **Missing** · **Unlocks**

T6 Serving oracles / A/B → **C2**
Exists: unit/golden/Alive2 · **FlashInfer-Bench**+`apply()` · VibeServe
Missing: whole-program / GPU-race / FP; multi-month default-path A/B

T7 Multi-IR corpora → **Selectors**
Exists: Meta LLM Compiler · **KernelBook**→TritonRL · DRTriton
Missing: versioned MLIR / Tile / StableHLO + failed / miscompile negatives

T8 Benchmark ladder → **C2, C9**
Exists: KernelBench(-X) · FlashInfer-Bench serving-kernel rung
Missing: full IR→kernel→fused→serving + cost-to-compile

T9 Provenance / HITL → **C7**
Exists: Magellan reviewable C++ · Archer oracle review
Missing: CODEOWNERS + signed admit records + sandbox for proposals

T10 Workflow compile / freeze → **Horiz. B**
Exists: FlowCompile · Auto · AgentFlow · VibeServe (early)
Missing: shared agent-graph IR + fail-closed CI → Horizon B

Technical Prediction — What Each Checkpoint Needs

C1 ← T4
Magellan-class synthesis *or* EmitC-MLGO customer default on named apps

C2 ← T3 + T6 + T8
Replayable artifacts + serving oracles + ladder
FlashInfer-Bench = pressure, not settlement

C5 ← T1 + T3
Typed interfaces + freeze-before-serve named in release notes

C3 / C6 ← T1 + T2
Constrained tools + admit/fallback (lean: hybrid advisory)

C9 ← T5 + T8
Coverage→perf agents + dialect sinks; need 2nd-vendor TritorX-class path

C10 ← T5
ISA/dialect *proposals only*; humans + chip-design tools own tape-out

Technical Prediction — Critical Missing Parts Now

Highest-leverage bets to accelerate Horizon A

1. Money-grade oracle stack (T2+T6) w/o → demos
local formal → shape-grid diff → serving statistics → staged rollout

2. Replayable artifact contract (T3) w/o → CI rejects
control files / kernels / heuristics with cache keys + golden replay

3. Portable agent compile interface (T1) w/o → re-glue
summaries · actions · admit records across MLIR / Triton / Tile / StableHLO

4. Open ladder + multi-IR data (T7+T8) w/o → incomparable
correctness × speed × $/compile + negative (failed) examples

Survey lean: T1–T5 ship as product surfaces; T6–T10 equally first-class — mostly outside classical lowering.

Org Adoption Questions

1. Online (CompileIQ) vs Offline (Magellan) — which matches your release model?
2. Oracle stack: Alive2 / golden / serving A/B — who owns false negatives?
3. Agent contract surface: LLVM / MLIR / Triton / StableHLO / Tile?
4. How do you cache/regress traces across compiler *and* model upgrades?
5. Max $/build for a median X% win? Named maintainer per admitted artifact?

Working stance until conflicts settle

1. Hybrid: agents search; compilers lower (not replace)
2. Magellan $\|$ MLGO — parallel bets (C1)
3. Discount single-number speedups (C2)
4. Constrained actions + strong oracles (C3)
5. Demote generic SCM AI (C7)
6. Coverage$\rightarrow$performance; no autonomous chip design (C9/C10)
7. $\sim$6–7 data-plane bands; plugins if not consolidated (A6/S6)
8. E2E-optimal-*seeking*: joint search under F (A7/S7)
9. Soft merge M1 by Horizon B; hard replace M3 not claimed (A8/C6)

Commercial checklist — handout

Architecture

typed tools + admit traces

version control $\gg$ dense memory $\gg$ scratchpad

state-machine plan for service targets

freeze before default-on

Trust

layered oracles

CODEOWNERS + provenance

joint version pins

A/B before "prod default"

Business

sell regressable artifacts + build-CI quota

customer owns outputs

coverage then performance service target

token budget per %gain

Discussion

Thank you — hybrid bet stays the ask

1. Which checkpoint flips your investment first — C1, C2, or C9?
2. Harder commercial bar: **oracle strength** or **replay/freeze**?
3. Building job (a), (b), or (d) first — what do you refuse online?

Ask again: is the hybrid bet right for your roadmap? Watch settlement signals — do not average disagreements.

Appendix

Evidence Store Snapshot

Publications digests · commercial product signals · open repositories.
Tiered for the agentic-compiler prediction — not a full catalog.

Appendix — evidence map

Publications

∼115 digests
one file per source

papers · blogs · talks
forge pages · minutes

★ = prediction-critical

Products

Commercial SKUs as *prediction signals*

Tier A shapes the future
Tier B = data-plane defaults

not a full SKU list

Repositories

GitHub / Gerrit / forge artifacts tiered A/B/C

A = agents reshape compile
C = generic forge AI only

not an exhaustive forge map

Use ★ digests + Tier A products/repos when a checkpoint or blocker is contested.

Appendix — Tier A commercial signals

Company offerings that shape what ships — agent loop / kernel surface / cloud agent

Google / DeepMind	AlphaEvolve Cloud (GA); Magellan + MLGO (heuristics *and* advisors)
NVIDIA	CompileIQ (+ agent-skills); CUDA Tile / Tile IR; TensorRT-LLM agent skills
AMD	GEAK (multi-agent Triton / Instinct kernel optimization)
Meta	LLM Compiler / KernelLLM; TritorX + KernelEvolve; Helion
FlashInfer	FlashInfer-Bench (serving-trace ladder + `apply()` into SGLang/vLLM)

Tier B baselines: TensorRT-LLM · Inductor · XLA/StableHLO · FlashInfer · Modular MAX · OpenVINO · Neur

Appendix — Tier A open repositories

Offline / review / heuristics
OpenEvolve · HeuriGym · Archer · Compiler-R1 · HintPilot · mlirAgent

Online / kernels / bring-up
ACCLAIM · CompileIQ · GEAK · KernelAgent · KernelBench(-X)
FlashInfer-Bench · AutoKernel · Helion · TritorX / KernelEvolve

Tiers: A = agents + domain oracles change heuristics/kernels/knobs/review
B = data-plane hosts · C = generic forge AI (demoted for prediction)

Appendix — prediction-critical digests (★)

Highest-signal sources — mechanisms, not headlines

Vision / funded	Compiler 2.0 Ken Kennedy (MIT) · DARPA MOCHA / Aarno
Offline / online	Magellan · ACCLAIM · EmitC-MLGO June 2026 PoR
Bring-up / codesign	TritorX · KernelEvolve · Ascend diagnosis · KForge
Kernels / ladder / data	CompileIQ · FlashInfer-Bench ★ · KernelBook ★ · AutoKernel · CuTeGen · Auto ·

Full digest index ∼115 entries. Technique map: SURVEY §5.8 T1–T10.

Appendix — publication groups

17 GPU kernels & inference **13** Agentic & RL **13** Source control & review
10 Classic DL compilers **10** Company infra **9** Forums & workshops
8 Surveys & vision **8** Foundation LLMs **6** MLGO & RL gyms
5 HW codesign & bring-up **4** Commercial products **4** Control-plane substrate

How digests are used

Mechanism first; demote generic forge AI; keep both sides — do not average
Update prediction only when Tier A / ★ evidence moves